STEM Website - Complete Work Summary

Implementation history, current repository state, verification evidence, and remaining roadmap

Repository

RoboLogic STEM Lab / STEM Website. Review and implementation scope was limited to the provided repository. Friend-owned work, new game modules, and unrequested visual redesigns were kept out of scope.

Current state at report time

Branch: feature/platform-stabilization. The branch is aligned with origin at commit b13f4c5. The working tree contains uncommitted documentation corrections prepared during the latest planning/implementation turn. The later shared-adapter factory and game-page modularization were not completed because implementation was paused for this summary.

High-level outcome

Area	Result
Backend authority	Server-side validators for RoboMaze and Physics; client-trusted legacy writes revoked.
Idempotency	Attempt receipts and idempotent submit wrappers added; direct authoritative submit RPC access later revoked.
Authentication	Protected gameplay fails closed when authentication is unavailable; shared auth/progress APIs remain centralized.
Tutor reliability	Retry, timeout, provider states, quota refund, safe fallback, and privacy-safe diagnostics added.
Tests and CI	Repository checks, syntax checks, secret scanning, Deno configuration, and 32 automated Node tests.
Security	Pinned Supabase browser client, CSP/referrer policies, RLS/RPC hardening, and no secret exposure in tracked code.

1. Repository and delivery foundation

- Created and used the feature/platform-stabilization branch while preserving existing work.

- Added repository documentation, Supabase configuration guidance, CI quality checks, required-file checks, JavaScript syntax checks, and secret-pattern scanning.

- Fixed the GitHub Deno test configuration by adding Deno npm dependency resolution and using explicit Deno.test declarations.

- Pinned the browser Supabase client version across the HTML entry pages.

- Added fail-closed authentication behavior to both game adapters.

2. Backend stabilization

The backend hardening work is the most complete part of the project. The migration sequence now reaches 013:

- Migration 011 revokes legacy client-trusted RPC access for record_game_attempt, record_student_achievement, and consume_tutor_quota.

- Migration 012 adds attempt_receipts and idempotent RoboMaze and Physics submit wrappers keyed by user, game, and attempt ID.

- Migration 013 revokes direct authenticated access to the non-idempotent authoritative submit procedures after the compatibility window.

- The user confirmed the final Supabase privilege state: direct submit procedures false; idempotent wrappers true.

- The frontend progress API no longer falls back to the legacy client-trusted attempt RPC.

3. Tutor and operational reliability

- Tutor provider failures are categorized as AI_RATE_LIMITED, AI_TEMPORARILY_UNAVAILABLE, or AI_INVALID_RESPONSE.

- Gemini calls use limited retry with exponential backoff and timeout handling.

- Quota reservations are refunded when provider or response validation fails.

- Browser diagnostics record only tutor state and latency. Edge Function logs record request ID, game ID, level, provider status when relevant, result state, and latency; prompts and student data are excluded.

4. Security and quality work

- Added CSP and strict-origin-when-cross-origin referrer policies to the entry pages. The current CSP still permits unsafe-inline because the existing pages contain inline scripts; removing that allowance is a future refactor gate.

- Added web-quality verification for page metadata, local asset references, pinned Supabase CDN usage, security metadata, and direct platform-call bypasses.

- CI syntax-checks JavaScript in js, PhysicsSim, and RoboMaze.

- Repository checks verify migration numbering, required tutor states, required files, and secret-pattern safety.

5. Verification evidence

The following checks were run against the current repository:

- node scripts/verify-repo.mjs - passed

- node scripts/verify-web-quality.mjs - passed

- JavaScript syntax checks - passed

- Automated tests - 32 passed, 0 failed

- Migration numbering and backend contract tests - passed

- User-confirmed hosted Supabase RPC privilege cleanup - passed

6. Documentation updates currently prepared

- README deployment instructions are being corrected from migrations 001-010 to 001-013.

- Backend verification SQL is being updated to include idempotent wrappers and the final privilege expectations.

- The repository verifier now rejects outdated migration-range and server-authority wording.

7. Work intentionally not completed

- The shared adapter factory was planned but not completed in the paused turn; current adapters still contain duplicated platform behavior.

- RoboMaze and Physics remain page-sized applications with large inline controllers.

- Inline event handlers and unsafe-inline CSP reliance remain.

- Playwright browser tests have not yet been added.

- Frontend/seed level parity checks have not yet been added.

- package-lock.json has not yet been generated.

- No new game modules were implemented.

8. Remaining roadmap

Next

Finish documentation corrections, then implement the shared adapter factory and add delegation tests.

After that

Modularize RoboMaze into explicit controller, UI, simulation, levels, and adapter files. Preserve behavior and replace inline handlers with event listeners.

Then

Apply the same modularization to Physics, followed by frontend/seed parity checks.

Verification

Add Playwright journeys for login, protected access, level completion, unlocks, retries, duplicate submissions, session expiry, tutor fallback, and refresh.

Release gate

Generate package-lock.json, switch CI to npm ci, tighten CSP after inline scripts are removed, confirm GitHub Actions, and complete browser plus hosted Supabase verification.

Future games

Only after both current games pass the browser, parity, and release gates should new game modules begin.

Beginner handoff

No new Supabase migration is required for the next frontend work. When a future migration is introduced, review it locally first, apply it in Supabase SQL Editor in order, run the verification SQL, and only deploy an Edge Function if its source changed. Never share service-role keys, Gemini keys, access tokens, or full environment files.